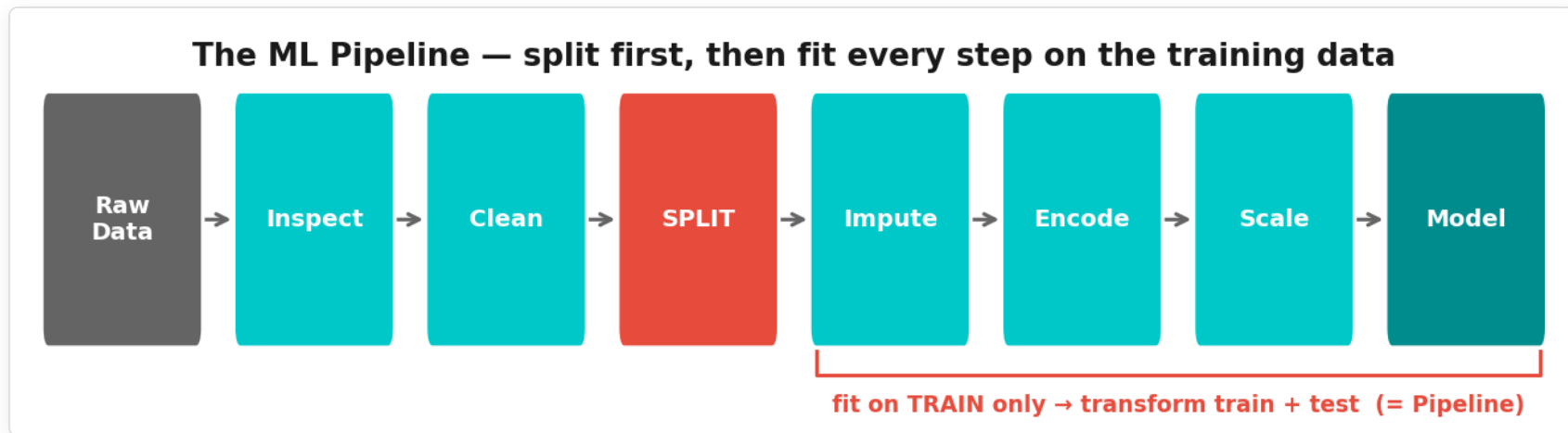


Data Selection, Cleaning & Preparing

Applied Machine Learning — Session 1, Chapter 2

The ML Pipeline



Garbage in — garbage out. This chapter = the data cleaning step.

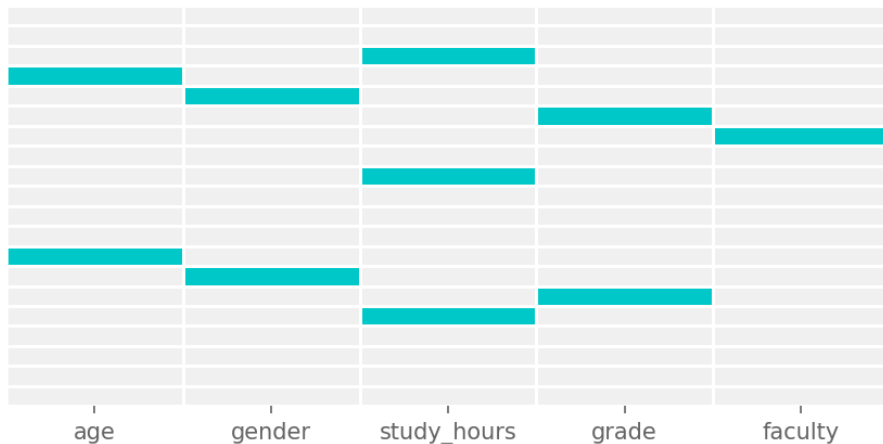
What Can Go Wrong?

- Missing values (NaN , empty strings, -999)
- Wrong data types (age as string)
- Inconsistent categories ("MaLe" , "maLe" , "M")
- Impossible values (age = 999 , salary = -1)
- Duplicate rows
- Mixed scales (salary in thousands vs. age in years)
- A feature that secretly *is* the answer (**target leakage**)

Missing Values — Detection

```
df.isnull().sum()      # count per column
df.isnull().mean()     # proportion per column
df.duplicated().sum()  # while we're at it: duplicate rows
```

Missing Values (teal = missing)

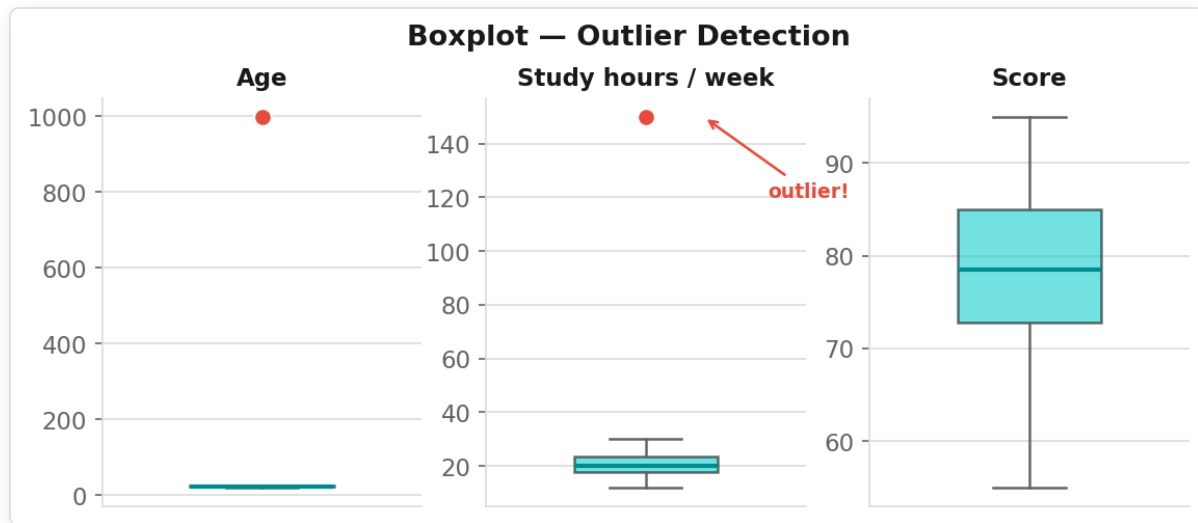


Missing Values — Strategies

Strategy	When to use
Drop rows	Few missing (< 5 %), random missingness
Drop column	Mostly missing (> 50 %)
Fill with mean	Numerical, no outliers
Fill with median	Numerical, with outliers (robust)
Fill with mode	Categorical
Fill with constant	Domain knowledge (e.g. "Unknown")

⚠ The fill value is a **statistic** → compute it on the **training set** only (`SimpleImputer` does exactly that).

Outliers



Detection: boxplot • IQR rule ($Q1 - 1.5 \cdot IQR$ / $Q3 + 1.5 \cdot IQR$) — bounds from **train**

Treatment: Remove (if clearly an error) • Cap/clip • Log-transform • **Keep** (if real signal)

Feature Types & Encoding

Type	Example	Ready for ML?
Numerical continuous	age, salary	after scaling
Numerical discrete	# children	usually yes
Categorical nominal	city, colour	one-hot encoding
Categorical ordinal	S / M / L / XL	ordered integer encoding
Binary	yes / no	0 / 1

One-Hot in Practice

One-Hot Encoding — faculty column

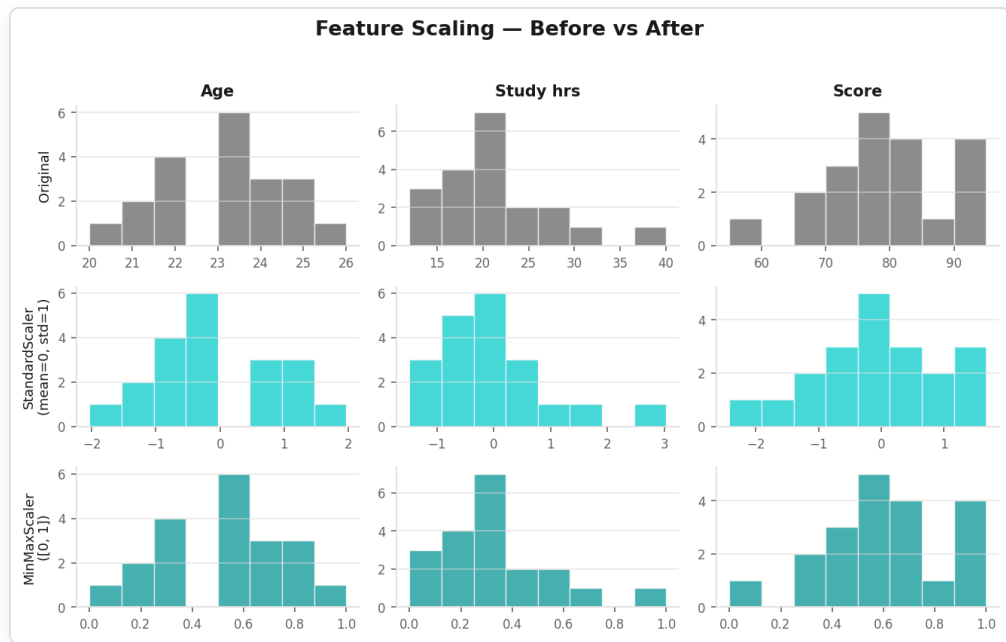
faculty	faculty_Arts	faculty_Engineering	faculty_Science
Science	0	0	1
Arts	1	0	0
Engineering	0	1	0
Arts	1	0	0
Science	0	0	1

```
pd.get_dummies(df, columns=['city'])  
OneHotEncoder(handle_unknown='ignore')
```

```
# quick & dirty (pandas)  
# sklearn – learns the categories on TRAIN,  
# unseen test category → all zeros, no crash
```

⚠ 100 cities → 100 new columns. High cardinality → target encoding / embeddings (not in this course).

Feature Scaling



StandardScaler → mean 0, std 1

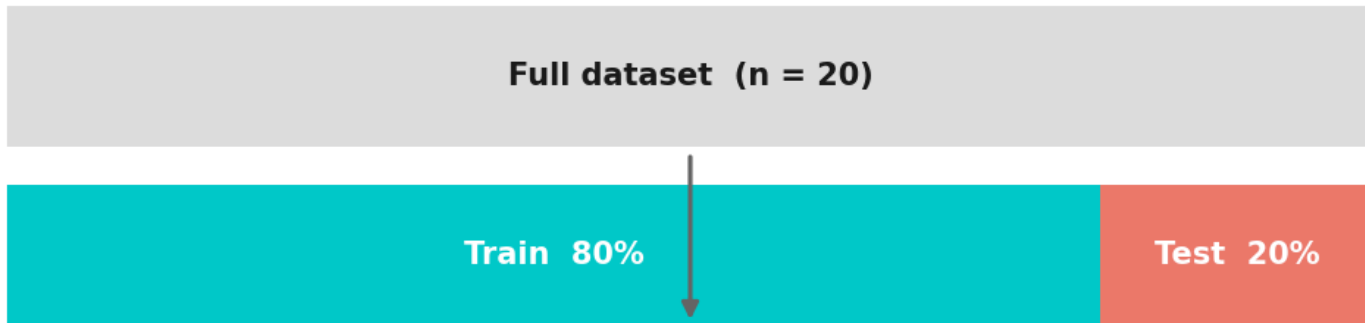
MinMaxScaler → [0, 1]

Needed for **distance- and gradient-based** models (KNN — next chapter!, linear/logistic regression, SVM, neural nets).

Tree-based models (Decision Tree, Random Forest) → no scaling needed.

Train / Test Split

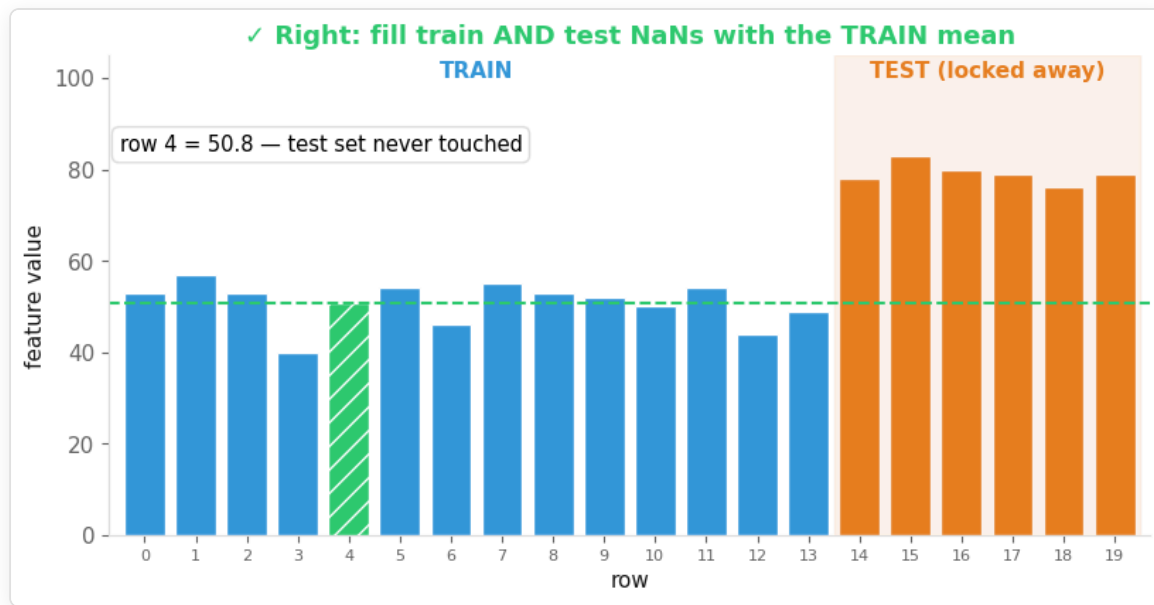
Train / Test Split (random_state=42, stratify=y)



```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42,
    stratify=y)      # ← classification only! keeps class proportions. Omit for regression.
```

Always set `random_state` — reproducible splits. The test set is locked away until the very end.

Data Leakage ⚠️ — the #1 beginner mistake



The Right Order

✗ Wrong order — test data leaks into the statistics



Rule: anything that computes a statistic (mean, median, min/max, std, IQR, categories) → fit on train, transform both

✓ Right order — everything is fit on the training set only



```
pipe = Pipeline([('imputer', SimpleImputer(strategy='median')), ('scaler', StandardScaler())])
pipe.fit(X_train)                                # statistics from TRAIN only
X_test_ready = pipe.transform(X_test)           # same statistics applied to test
```

Quick Check — Which of These Steps Leak?

You have `df` with 1 000 rows. Steps, in this order:

1. `df['city'] = df['city'].str.lower()`
2. `df['age'] = df['age'].fillna(df['age'].median())`
3. `train_test_split(...)`
4. `scaler.fit(X_train); scaler.transform(X_test)`
5. `df.drop_duplicates()` before the split
6. Dropping the column `"final_invoice_amount"` when predicting `"will_customer_buy"`

Quick Check — Which of These Steps Leak?

You have `df` with 1 000 rows. Steps, in this order:

1. `df['city'] = df['city'].str.lower()`
2. `df['age'] = df['age'].fillna(df['age'].median())`
3. `train_test_split(...)`
4. `scaler.fit(X_train); scaler.transform(X_test)`
5. `df.drop_duplicates()` before the split
6. Dropping the column `"final_invoice_amount"` when predicting `"will_customer_buy"`

Leaks: 2 (median over all rows — must be after the split, fit on train). **Safe:** 1, 3, 4, 5. 6 is *removing* target leakage — the invoice only exists after the purchase.

Now: Live Demo, Then Exercises

Demo (~8 min): `02-examples/ch02_data_cleaning_examples.ipynb` messy student survey → duplicates, text fixes, target leakage (`grade`), split, `SimpleImputer`, IQR, encoding, scaling → `ColumnTransformer` + `Pipeline` in 10 lines

Exercises (~10 min): `03-exercises/ch02_data_cleaning_exercises.ipynb` messy housing data • Task 1 fixes → Task 2 split → Task 3 impute → Task 4 `ColumnTransformer` • Bonus: IQR, mean vs median, Z-score, correlation

Key Takeaways

- Real data is always messy — cleaning is non-negotiable
- Deterministic fixes (text, duplicates, impossible values) → before the split
- **Split first**, then impute / clip / encode / scale — **fit on train, transform both**
- Ask: does any feature secretly contain the answer? (target leakage)
- `Pipeline` + `ColumnTransformer` make the right order automatic

Bonus — Z-Score Outlier Rule

- Standardise: $z = (x - \text{mean}) / \text{std}$ (mean/std from **train**)
- Flag $|z| > 3$ as outliers ($\approx 0.3\%$ of a normal distribution)
- Sensitive to the very outliers it looks for (they inflate the std) $\rightarrow$ the IQR rule is usually the more robust default
- Try it: exercise **Bonus C**

Next: Chapter 3

Introduction to Supervised Learning

"Now that our data is clean, let's teach a machine to learn from it."